Win32 App Uninstall & Removal Failure Remediation (Microsoft Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What you get . . . 4

Business impact . . . 4

Bottom line . . . 5

Solution Overview 5

The problem this solves . . . 5

Scope — what this solution covers, and what it does not . . . 5

How Intune actually processes an Uninstall intent . . . 6

The diagnostic model . . . 7

The objects this solution creates . . . 7

The standard deployment model . . . 8

What is in this solution . . . 8

Requirements 8

R1. An Intune role that can read apps, assignments, and device install status . . . 8

R2. An Intune role that can modify apps and assignments (remediation only) . . . 9

R3. Microsoft Graph PowerShell SDK access with the correct scopes . . . 9

R4. Local administrator access to a representative failing device . . . 10

R5. Access to the Intune Management Extension logs . . . 10

R6. The app's original packaging record — or the ability to reconstruct it . . . 11

R7. A pilot device group and a change window . . . 11

R8. PowerShell 5.1 or PowerShell 7 . . . 12

Government (GCC / GCC High / DoD) note . . . 12

Global Requirements — Intune Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 13

G2. Microsoft Intune subscription . . . 13

G3. Administrative roles . . . 13

G4. Device enrollment in Intune . . . 13

G5. Pilot security group . . . 13

G6. Supported Windows baseline . . . 13

G7. Network access to Microsoft cloud endpoints . . . 13

Diagnostic Playbook — the Win32 App Removal Failure Tree 14

Step 0 — Establish the facts (always do this first) . . . 14

Branch A — Uninstall does nothing, no error (assignment intent conflict) . . . 15

Branch B — Uninstall reported success, but the app is still there (detection said "gone" pre-emptively) . . . 17
Branch C — Uninstall runs forever / never reports done (detection still matches after removal) . . . 17
Branch D — "Uninstall failed (1605)" (wrong or stale MSI ProductCode) . . . 18
Branch E — "Uninstall failed (1603 / other non-zero)" (wrong uninstall command or EXE switches) 19
Branch F — Works for some, fails for others (per-user vs per-machine / SYSTEM context) . . . 20
Branch G — "Uninstall failed (1618)", file-in-use, or reboot required (app running / pending reboot) 20
Branch H — Superseded app won't come off during an upgrade (supersedence uninstall command) 21
Branch I — Device never reports / state "Unknown" / app object deleted mid-removal (reporting & lifecycle) . . . 22
After any fix — force a sync and re-validate the whole removal . . . 23
Testing 23
Gate 1 — Gone from the device (the software is physically removed) . . . 23
Gate 2 — Uninstall command returned success . . . 24
Gate 3 — Intune agrees it is gone (state = Not installed) . . . 24
Gate 4 — It stays gone (no re-attempt on the next cycle) . . . 25
Full-removal regression check (run on the pilot before expanding) . . . 25
Troubleshooting 25
Connecting to Microsoft Graph . . . 25
Running the on-device scripts . . . 26
Reading the IME logs . . . 27
Interpreting states . . . 27
Remediation script . . . 28
When the method stalls . . . 28
Best Practices 28
Author the uninstall command at package time — not at removal time . . . 29
Detection rules decide removal — make them match reality on both ends . . . 29
Keep intents unambiguous — never target install and uninstall together . . . 29
Map return codes so benign outcomes are not failures . . . 30
Stop the app before removing it . . . 30
Remove the right way — assignment first, delete the object last . . . 30
Operational hygiene . . . 30
Deliverables Summary 31
Documents . . . 31
PowerShell scripts . . . 31
How the pieces fit . . . 32
Appendix A — PowerShell Scripts 34

Executive Summary

The problem

You told Microsoft Intune to remove a Win32 application, and it did not come off. The console reports **Uninstall failed**, or a raw code like **1605**, or the device sits at **Pending** — or, worse, Intune reports the app as **removed** while it is plainly still installed on hundreds of machines. The team retries, re-syncs, re-assigns, and eventually deletes the app object to "force" it, which strands the software on every device that had not yet processed the removal. Hours are burned, licences keep being consumed, and non-compliant or unwanted software lingers on the fleet.

Removal is not installation in reverse. Intune's Uninstall intent has its own delivery path, its own exit codes, and its own failure modes — almost all of them invisible in the console. Guessing does not fix them.

The solution

This is a **branch-based diagnostic playbook for the Win32 app removal path specifically**. It explains how Intune actually processes an **Uninstall** intent — how it reconciles conflicting intents (`Required` silently beats `Uninstall`), how **detection rules gate the removal on both ends** (deciding whether the uninstall runs *and* whether it is judged successful), and how the Intune Management Extension executes the uninstall command and maps its exit code. From that model it gives you a **Step 0 fact-finding routine** and **nine labelled branches (A–I)** — intent conflict, detection-said-gone, detection-loop, stale ProductCode (1605), wrong EXE switches (1603), per-user vs per-machine context, app-in-use (1618), supersedence removal, and reporting/lifecycle — each with **symptom** → **why** → **exact fix** → **re-validate**. Five PowerShell scripts read the real state from the device and Microsoft Graph and make safe, gated fixes.

What you get

- A precise **model of the Uninstall intent** — intent reconciliation, detection-gated removal, exit-code mapping, install context — so "uninstall failed" becomes a named cause.
- A **Diagnostic Playbook** of nine removal-failure branches with exact console navigation, registry paths, exit-code decoding, and the IME log lines to search for.
- **Five PowerShell scripts** — read-only on-device Discovery (real ProductCode + uninstall string + cache state), an assignment-conflict detector, a Graph uninstall-status Reporter, an IME LogParser, and a **-WhatIf**-gated Remediation that corrects the uninstall command, detection rule, or conflicting assignment.
- **Four removal gates** that prove the software is physically gone, that Intune agrees, and that it stays gone — plus **Best Practices** that stop these failures at packaging time.

Business impact

- **Faster resolution:** removal failures traced to their real cause in minutes, not iterated on for days.

• **Clean fleet:** unwanted, non-compliant, or licence-costing software is actually removed and proven gone — not just marked gone.

• **Lower risk:** every removal is piloted and validated before the fleet; remediation is gated and reversible in preview.

• **Retained capability:** the method and scripts stay with your team and work for every future app retirement.

Bottom line

Every failed Win32 uninstall is traced to its real cause — intent conflict, detection, ProductCode drift, command switches, install context, app-in-use, supersedence, or reporting — and fixed, with a repeatable method and scripts you keep, at a fraction of the cost of ad-hoc troubleshooting or outside consulting.

Solution Overview

The problem this solves

You assigned a Win32 app in Microsoft Intune with the **Uninstall** intent. The app does not come off. Intune reports **Uninstall failed**, or **Failed (1605)**, or the device sits at **Pending** forever, or — worst of all — Intune reports the app as **Installed** even though you asked for it to be removed and it *is* still there. Someone re-assigns it, someone else deletes the app object, and a week later the software is still on 400 machines.

Removal is not "install, backwards." It is its own delivery path with its own failure modes, and almost every one of them is invisible in the Intune console. This solution is a **branch-based diagnostic playbook for the uninstall path specifically** — how Intune actually processes an Uninstall intent, why detection rules govern removal just as tightly as they govern installation, what the uninstall command line really has to be, and how to prove the software is gone.

Scope — what this solution covers, and what it does not

In scope (the removal path):

• The **Uninstall assignment intent** on Win32 apps, and how the Intune Management Extension (IME) acts on it.

• **Assignment intent conflict** — a device targeted with **Required** (install) *and* **Uninstall** through two different groups. This is the single most common root cause of "uninstall does nothing."

• **Detection rules on the removal path** — the two opposite failure modes they cause.

• The **uninstall command line**: MSI **ProductCode** drift between versions, stale ProductCodes, EXE silent-uninstall switches, quoting/escaping errors, and uninstall strings read out of the registry.

• **Install context** — per-machine (`HKLM`) vs per-user (`HKCU`), and SYSTEM vs user execution.

• The **missing or corrupt Windows Installer cache** (`C:\Windows\Installer`) and what to do when the

original installer no longer exists anywhere.

- Apps that **cannot be removed while running** — file-in-use, locked services, pending reboot.
- **Uninstall-specific exit codes** (0, 1605, 1614, 1618, 1641, 3010, 1603, 1612, 1639 and friends)

and which of them must be re-mapped in the app's **Return codes** table.

- **Supersedence-driven** uninstall failure — the superseded app's own uninstall command is wrong.
- Devices that **never report**, stale check-in, and the app object deleted **before** removal

completed.

- **IME log reading** (`IntuneManagementExtension.log`, `AppWorkload.log`, `AgentExecutor.log`) —

exactly what to search for on the uninstall path.

- **Last-resort removal** paths, and how to **prove** the app is actually gone.

Out of scope (deliberately): install-path diagnosis — "Not applicable," requirement-rule failures, content download failures, dependency resolution, and Enrollment Status Page blocks. Those are the *install* pipeline. This solution starts where the install pipeline ends.

How Intune actually processes an Uninstall intent

Understanding this sequence is what makes the playbook work. The IME is the agent that runs Win32 app work on Windows; it polls Intune roughly **every 8 hours**, and on device check-in, sync, or service restart.

1. **Intent resolution.** The IME receives the full list of Win32 app assignments that apply to this

device and this user. Each assignment carries an intent: **Required**, **Available**, or **Uninstall**. If more than one assignment targets the same app for the same device, **the intents are reconciled — and** `Required` **(install) wins over** `Uninstall`**.** A device in both an install group and an uninstall group therefore keeps the app, silently, forever. Nothing errors.

1. **Detection evaluation — BEFORE the uninstall runs.** The IME evaluates the app's **detection

rules **first. If detection says the app is** not present**, there is nothing to remove: the IME marks the app as uninstalled/not installed and** never runs the uninstall command line at all**.

1. **Uninstall execution.** If detection says the app **is** present, the IME runs the app's

Uninstall command — from `Properties > Program > Uninstall command` — in the app's configured **Install behavior** context (**System** or **User**). The IME captures the process **exit code**.

1. **Exit-code mapping.** The returned code is mapped through the app's **Return codes** table to

Success, Failure, Soft reboot, Hard reboot, or Retry. An unmapped code defaults to **Failure** even when the removal actually worked.

1. **Detection evaluation — AFTER the uninstall.** The IME re-runs the **same detection rules**.

Removal is only reported successful when detection now returns **not detected**. If detection still matches — a leftover registry key, a leftover file, an over-broad version operator — Intune treats the removal as failed and **retries the uninstall on every cycle, indefinitely**.

1. **Reporting.** The device reports status back; it appears under the app's **Device install

status** blade and in Graph as an `installState` of `uninstallFailed`, `notInstalled`, `installed`, `pendingInstall`, or `unknown`.

The two facts that explain most uninstall tickets:

> ***Detection is the gate on both ends of the removal.*** *It decides whether the uninstall command runs *at all*, and it decides whether the removal is judged successful. A detection rule that is wrong in one direction makes uninstall a no-op; wrong in the other direction, it makes uninstall an infinite retry loop.*
>
> ***Required beats Uninstall.*** *Intent conflicts do not raise an error anywhere in the console. You must go looking for them.*

The diagnostic model

Three moves, always in this order:

1. **Establish the facts (Step 0).** Determine the *reported* state, the *actual* on-device state,

and every assignment that touches this app. Never start fixing before you know all three; most wasted hours come from fixing a command line when the real problem was an assignment.

1. **Pick the one branch that owns your symptom.** The playbook in `03-implementation.md` is a set of

labelled branches (A–I). Each branch gives **symptom** → **why it happens** → **the exact fix** → **how to re-validate**. Work one branch at a time.

1. **Prove removal, then expand.** Removal is only "done" when the app is gone from the device *and*

Intune agrees it is gone *and* it does not come back on the next cycle. The gates in `05-testing.md` enforce all three.

The objects this solution creates

These exact names are used consistently in every step, test, and script in this package:

Object	Exact name	What it is
Entra security group (**Assigned**)	`APPUNIN – Uninstall – Pilot`	The small pilot group that receives the Uninstall assignment first.
Entra security group (**Assigned**)	`APPUNIN – Uninstall – Production`	The fleet group that receives the Uninstall assignment after the pilot passes.
Entra security group (**Assigned**)	`APPUNIN – Uninstall – Exclusions`	Devices explicitly excluded from removal (break-fix, executives, lab).
Intune device filter	`APPUNIN – Windows Only`	Filter used on the Uninstall assignment so it applies only to Windows 10/11 devices.

Object	Exact name	What it is
Evidence folder (on device)	`C:\ProgramData\APPUNIN\Evidence`	Where you save the verbose MSI log and the discovery output for the failing device.

The standard deployment model

This solution follows the same model as every solution in this category: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** For a removal campaign that means: establish the facts, correct the app object, prove removal on `APPUNIN – Uninstall – Pilot`, then move the assignment to `APPUNIN – Uninstall – Production`.

What is in this solution

- **This overview** — how the Uninstall intent is processed and the diagnostic model.
- **Requirements** (`02-requirements.md`) — roles, Graph scopes, device access, each with a verify step and a fix if missing.
- **Diagnostic Playbook** (`03-implementation.md`) — Step 0 plus branches **A–I**, the core of the package.
- **Testing** (`05-testing.md`) — the four pass/fail removal gates.
- **Troubleshooting** (`06-troubleshooting.md`) — problems with the method, scripts, and access.
- **Best practices** (`07-best-practices.md`) — how to author apps so removal works the first time.
- **Deliverables summary** (`08-deliverables-summary.md`) and the **executive summary**.
- **Five PowerShell scripts** — Discovery, Validation, Report, LogParser, and Remediation.

Requirements

Confirm every item below **before** you start the Diagnostic Playbook. The category-level prerequisites (Entra tenant, Intune subscription, enrolled devices, supported Windows baseline, network access) are in `global-requirements-intune-solutions.md` and are assumed here. This file covers what is specific to **diagnosing and fixing Win32 app removal**.

Each requirement is written as **Description / Why required / How to verify / How to implement if missing.**

R1. An Intune role that can read apps, assignments, and device install status

Description: An Entra/Intune role granting read access to **Apps**, their **Assignments**, their **Properties** (Program, Detection rules, Return codes), and per-device **Device install status**. **Intune Administrator** covers everything in this solution. For the read-only scripts, **Help Desk Operator** — or a custom role with

Managed apps → **Read** and **Organization** → **Read** — is sufficient.

Why required: Every branch of the playbook begins by reading the app's uninstall command, detection rules, return-code table, and the per-device state. Without app read rights the console shows nothing and the scripts return empty results.

How to verify: Sign in to `https://intune.microsoft.com`. Go to **Intune admin center > Apps > All apps**, open any Win32 app, and confirm you can open both **Properties** and **Device install status**. Then go to **Intune admin center > Tenant administration > Roles > All roles** and confirm your role assignment is listed.

How to implement if missing: Go to **Intune admin center > Tenant administration > Roles > All roles**, select **Help Desk Operator**, select **Assignments**, click **Assign**, name the assignment `APPUNIN – Read Only Operators`, add your account under **Admin Groups**, set the **Scope (Groups)** to the groups you support, and click **Create**.

R2. An Intune role that can modify apps and assignments (remediation only)

Description: Write access to apps and their assignments — **Intune Administrator**, or a custom role granting **Managed apps** → **Read, Update, Assign**.

Why required: Fixing a removal failure means editing the app's **Uninstall command**, its **Detection rules**, or its **Return codes**, and removing a conflicting **Required** assignment. The read-only Discovery, Validation, Report, and LogParser scripts never need this; only `APPUNIN-Remediation.ps1` does.

How to verify: Go to **Intune admin center > Apps > All apps > _<your app>_ > Properties**, click **Edit** next to **Program**, and confirm the **Uninstall command** field is editable and the **Review + save** button is available. Cancel without saving.

How to implement if missing: Go to **Intune admin center > Tenant administration > Roles > All roles > Create**, choose **Windows (Custom role)**, name it `APPUNIN – Removal Operators`, grant **Managed apps** → **Read**, **Update**, **Assign**, click **Create**, then open the role and use **Assignments > Assign** to add only the operators who will apply fixes.

R3. Microsoft Graph PowerShell SDK access with the correct scopes

Description: Delegated Microsoft Graph permissions for the five scripts:

Script	Scopes needed
`APPUNIN-Discovery.ps1`	*(none — runs locally on the device, no Graph)*
`APPUNIN-Validation.ps1`	`DeviceManagementApps.Read.All`, `Group.Read.All`, `DeviceManagementManagedDevices.Read.All`
`APPUNIN-Report.ps1`	`DeviceManagementApps.Read.All`, `DeviceManagementManagedDevices.Read.All`
`APPUNIN-LogParser.ps1`	*(none — parses log files, no Graph)*
`APPUNIN-Remediation.ps1`	`DeviceManagementApps.ReadWrite.All`

Why required: The assignment-conflict detector must read every assignment on the app *and* resolve group membership to see whether one device sits in both an install group and an uninstall group. The reporter reads per-device uninstall state and error codes. The remediation script writes the corrected uninstall command, detection rule, or assignment.

How to verify: Run `Get-Module Microsoft.Graph -ListAvailable` and confirm a version is listed. Then run `Connect-MgGraph -Scopes "DeviceManagementApps.Read.All"`, sign in, and run `Get-MgContext` — confirm the **Scopes** property contains the scope you requested.

How to implement if missing: Install the SDK with `Install-Module Microsoft.Graph -Scope CurrentUser`. Each script calls `Connect-MgGraph` with the scopes it needs and prompts for consent on first run. If your tenant blocks user consent, have a Global Administrator grant admin consent once at **Entra admin center > Applications > Enterprise applications > Microsoft Graph Command Line Tools > Permissions > Grant admin consent**.

> ***US Government tenants (GCC High / DoD):*** *the scripts accept a* `-GraphEnvironment` *parameter. Pass* `-GraphEnvironment USGov` *for GCC High or* `-GraphEnvironment USGovDoD` *for DoD, which makes the script call* `Connect-MgGraph -Environment USGov` *(or* `USGovDoD`*) and target* `https://graph.microsoft.us` *instead of* `https://graph.microsoft.com`*. The admin console is* `https://endpoint.microsoft.us`*. See the note at the end of this file.*

R4. Local administrator access to a representative failing device

Description: Local administrator rights (or the ability to run as **SYSTEM**) on at least one enrolled Windows device that reproduces the removal failure.

Why required: The truth about a removal failure lives on the device, not in the console. You need to read the uninstall registry keys, confirm whether the cached MSI still exists, run the uninstall command by hand, and capture a verbose MSI log. `APPUNIN-Discovery.ps1` reads all of that and must run elevated on the device.

How to verify: On the device, open PowerShell as administrator and run `Test-Path 'C:\Windows\Installer'` (expect `True`) and `Get-ChildItem 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall'` (expect a list of keys).

How to implement if missing: Use **Intune admin center > Devices > Windows > Windows devices > _<device>_ > ... > Collect diagnostics** to pull the logs remotely, or add your account to the local Administrators group on a lab device that reproduces the failure. Create the evidence folder `C:\ProgramData\APPUNIN\Evidence` on that device — every branch of the playbook writes its evidence there.

R5. Access to the Intune Management Extension logs

Description: Read access to `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\` on the failing device — specifically `IntuneManagementExtension.log`, `AppWorkload.log`, and `AgentExecutor.log`.

Why required: These logs are the **only** place that records, in order: the intent the IME resolved for the app, the pre-uninstall detection result, the exact uninstall command line that was executed, the process

exit code it returned, and the post-uninstall detection result. Graph tells you *what* state the device is in; these logs tell you *why*.

How to verify: On the device, open `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` (note: `ProgramData` is hidden — type the full path into File Explorer's address bar). Confirm the file exists and has entries from the last 24 hours.

How to implement if missing: If the folder is absent, the Intune Management Extension is not installed — it installs automatically when a Win32 app or PowerShell script is assigned to the device. Assign any Win32 app to the device, wait for a sync, and re-check. To collect the logs without touching the device, use **Intune admin center > Devices > Windows > Windows devices > _<device>_ > ... > Collect diagnostics**, then download the resulting package from the **Device diagnostics** tab. Install **CMTrace** (or use Notepad) to read them.

R6. The app's original packaging record — or the ability to reconstruct it

Description: The record of how the app was packaged: the **MSI ProductCode** (per version), the silent uninstall switches for an EXE installer, the install context (per-machine vs per-user), and where the source installer is archived.

Why required: Branches C and E of the playbook turn on knowing the *correct* uninstall command. If the ProductCode drifted between versions, or the vendor's EXE uses a different switch for uninstall than for install, you cannot author a correct uninstall command from the console alone.

How to verify: Go to **Intune admin center > Apps > All apps > _<your app>_ > Properties > Program** and read the current **Uninstall command**. Then run `APPUNIN-Discovery.ps1` on a device that still has the app, and compare the command to the `UninstallString` / `QuietUninstallString` the installer actually registered. They must match.

How to implement if missing: Run `APPUNIN-Discovery.ps1` on any device that still has the app — it resolves the real ProductCode, the registered uninstall string, and whether the cached MSI is still present, which is enough to reconstruct a correct uninstall command. For EXE installers, get the silent-uninstall switch from the vendor's documentation; do not guess it. Record the result in the app's **Notes** field (**Properties > App information > Notes**) so it is never lost again.

R7. A pilot device group and a change window

Description: The **Assigned** Entra security group `APPUNIN – Uninstall – Pilot` containing 3–10 representative devices, plus an agreed window in which software may disappear from those devices.

Why required: Removal is destructive and, for most business software, not trivially reversible — user data, licences, and local configuration can go with it. Every fix in this playbook is proven on the pilot group before it reaches `APPUNIN – Uninstall – Production`.

How to verify: Go to **Entra admin center > Groups > All groups** and confirm `APPUNIN – Uninstall – Pilot` exists, that its **Membership type** is **Assigned**, and that it has members.

How to implement if missing: Go to **Entra admin center > Groups > All groups > New group**. Set **Group type** = **Security**, **Group name** = `APPUNIN – Uninstall – Pilot`, **Membership type** = **Assigned**, add your pilot devices under **Members**, and click **Create**. Repeat to create `APPUNIN –`

`Uninstall – Production` and `APPUNIN – Uninstall – Exclusions`.

R8. PowerShell 5.1 or PowerShell 7

Description: Windows PowerShell 5.1 (built into Windows) or PowerShell 7 on the operator workstation and on the device.

Why required: All five scripts target 5.1+ syntax. `APPUNIN-Discovery.ps1` and `APPUNIN-LogParser.ps1` run on the device with no external module dependency; the Graph scripts need the SDK from R3.

How to verify: Run `$PSVersionTable.PSVersion` and confirm **5.1** or **7.x**.

How to implement if missing: Windows 10/11 ships with 5.1. Install PowerShell 7 from the Microsoft Store or `winget install Microsoft.PowerShell`. Run the scripts with `powershell -ExecutionPolicy Bypass -File .\<script-name>.ps1`.

Government (GCC / GCC High / DoD) note

The removal behaviour documented in this solution — intent reconciliation, detection-gated uninstall, exit-code mapping, the Windows Installer registry and cache layout, and the IME log paths — is **Windows and Intune service behaviour and is the same in Government tenants**. What differs is **where you connect**, not what you do:

Surface	Commercial	US Government
Admin console	`https://intune.microsoft.com`	`https://endpoint.microsoft.us`
Microsoft Graph endpoint	`https://graph.microsoft.com`	`https://graph.microsoft.us`
`Connect-MgGraph` environment	`-Environment Global` (default)	`-Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD)
Sign-in authority	`login.microsoftonline.com`	`login.microsoftonline.us`

Every script in this package accepts `-GraphEnvironment Global | USGov | USGovDoD` and passes it through to `Connect-MgGraph`. Wherever this documentation shows `https://intune.microsoft.com`, Government operators substitute `https://endpoint.microsoft.us`; the blade names and navigation paths are otherwise identical. Feature-availability timing in GCC High and DoD can lag Commercial — **verify for GCC High** against Microsoft's US-sovereign-cloud documentation before assuming a newly released Intune app feature is present in your tenant.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Diagnostic Playbook — the Win32 App Removal Failure Tree

This is the core of the solution: a **branch-based diagnostic tree for the uninstall path**. Start from what you observe, follow the one branch that matches, run the exact checks, apply the exact fix, then re-validate against the gates in `05-testing.md`.

Golden rule of removal: Intune resolves the app's **intent** first (`Required` beats `Uninstall`), then it lets **detection** decide whether the uninstall command runs *and* whether the removal is judged successful. So the order of causes is: **Intent** → **Detection (pre)** → **Uninstall command** → **Install context** → **Exit-code mapping** → **Detection (post)** → **Reporting.** Find the *first* one that is wrong; everything after it is a symptom, not a cause.

Work **one branch at a time**. Never edit the uninstall command before you have confirmed the intent and the detection rule, because in most tickets the command line was fine all along.

Step 0 — Establish the facts (always do this first)

You need three facts before you touch anything: the **reported** state, the **actual** on-device state, and every **assignment** that touches this app.

1. **Read the reported state in Intune.** Go to

Intune admin center > Apps > All apps > _<your app>_ > Device install status. Find the affected device and note the exact **Status** text: *Uninstall failed*, *Pending*, *Not installed*, *Installed*, or *Unknown*. Also open **Apps > All apps > _<your app>_ > Overview** and note the **Uninstall failed** count — one device or the whole fleet points at different causes.

1. **Read the same state from Graph.** Run `APPUNIN-Validation.ps1 -AppNameLike "<your app>"`. It

returns, per device, the `installState` (`uninstallFailed`, `notInstalled`, `installed`, `pendingInstall`, `unknown`), the `installStateDetail`, and the raw `errorCode`. This is faster and more precise than the console for triage across many devices.

1. **List every assignment on the app — this is the step people skip.** Run

`APPUNIN-Validation.ps1` with `-CheckConflicts`, or go to **Intune admin center > Apps > All apps > _<your app>_ > Properties > Assignments**. Write down every group under **Required**, **Available for enrolled devices**, and **Uninstall**, and note whether your failing device (or its primary user) is in more than one of them. **A device in both a Required group and an Uninstall group is Branch A — stop here and go there.**

1. **Read the on-device truth.** On the failing device, in an elevated PowerShell, run

`APPUNIN-Discovery.ps1`. It reports, for the target app: the real **ProductCode**, the registered **UninstallString** and **QuietUninstallString**, whether the app is registered under **HKLM** (per-machine) or **HKCU** (per-user), whether the cached MSI still exists in `C:\Windows\Installer`, and whether the app's processes/services are currently running. Save the output to `C:\ProgramData\APPUNIN\Evidence`.

1. **Open the IME logs.** On the device, open

`C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` and `AppWorkload.log`, or run `APPUNIN-LogParser.ps1`. Search for the app's **display**

name or its **application id (GUID)**. In pipeline order you will see: the intent the IME resolved, the pre-uninstall detection result, the uninstall command line, the returned exit code, and the post-uninstall detection result. That sequence tells you which branch you are in.

Now pick the branch that matches your symptom.

If you observe…	Go to
Uninstall does nothing, no error, device stays *Installed*	**Branch A** (intent conflict) — check first
Uninstall reported *success* but the app is still on the device	**Branch B** (detection said "gone" — it lied)
Uninstall runs every cycle / never reports done, app *is* gone	**Branch C** (detection still matches after removal)
Uninstall failed (1605)	**Branch D** (wrong/stale ProductCode)
Uninstall failed (1603 / other non-zero) from an EXE or MSI	**Branch E** (wrong uninstall command / switches)
Removal works for some users, not others; app under `HKCU`	**Branch F** (per-user vs per-machine / SYSTEM context)
Uninstall failed (1618), file-in-use, or needs a reboot	**Branch G** (app running / reboot pending)
Superseded app won't come off during an upgrade	**Branch H** (supersedence uninstall command)
Device state *Unknown*, never reports, or app object was deleted	**Branch I** (reporting / lifecycle)

Branch A — Uninstall does nothing, no error (assignment intent conflict)

Symptom: You assigned the app with **Uninstall**, but it stays on the device and Intune keeps reporting it **Installed**. There is **no error** anywhere. Retrying and re-syncing change nothing.

Why it happens: The device (or its primary user) is in **two** assignment scopes for the same app — a **Required** (install) group *and* an **Uninstall** group — usually through different nested groups. When the IME reconciles the intents for a single app, `Required` **wins over** `Uninstall`**.** The app is therefore kept installed, and the uninstall command is never run. This is the single most common root cause of "uninstall does nothing," and it is invisible in the per-device status blade.

Checks:

1. Run `APPUNIN-Validation.ps1 -AppNameLike "<your app>" -CheckConflicts`. It lists every assignment

on the app, resolves group membership, and flags any device that lands in both an install intent and an uninstall intent.

1. Confirm manually: **Apps > All apps > _<your app>_ > Properties > Assignments**. Read the groups

under **Required** and **Available**, and the groups under **Uninstall**. Then, for the failing device, check **Devices > Windows > Windows devices > _<device>_ > Groups** (and the primary user's group

memberships) to see whether it is in a group on both sides. Watch for **nested** groups — a device can be in a Required group *transitively*.

1. In `IntuneManagementExtension.log`, search the app id: a line resolving the app to intent `Required` (or `EnforceInstall`) when you expected `Uninstall` confirms the conflict.

Exact fix:

1. Decide which intent is correct. To remove the app, the device must be **only** in an Uninstall scope.

1. Remove the device from the conflicting **Required**/**Available** group, **or** add it to an exclusion. Go to **Apps > All apps > _<your app>_ > Properties > Assignments > Edit**, and under **Required** add `APPUNIN – Uninstall – Exclusions`... no — instead, under **Required** click the group and use **Edit assignment**, then remove the failing device's group, or add the device to a group listed under the Required assignment's own **Excluded groups**. The cleanest pattern: maintain removal through the dedicated **Assigned** group `APPUNIN – Uninstall – Production` and ensure that group's devices are **not** members of any Required/Available group for this app.

1. Do **not** rely on the same group appearing under both Required and Uninstall — remove the Required assignment for the devices you want cleaned.

1. Click **Review + save**.

Re-validate: Force a sync on the device (**Branch-agnostic sync**, see the end of this file), wait a cycle, then re-run `APPUNIN-Validation.ps1`. The device should move to **Not installed**. Confirm in the log that the IME now resolves the app to intent **Uninstall** and runs the command.

Branch B — Uninstall reported success, but the app is still there (detection said "gone" pre-emptively)

Symptom: Intune reports the app **Not installed / uninstall succeeded**, but the software is plainly still on the device. The log shows the IME **never ran the uninstall command** — it went straight to "not detected, nothing to remove."

Why it happens: The IME evaluates detection **before** running the uninstall. If the detection rule returns **not detected** — because it points at the wrong file, an old version string, a per-user path that SYSTEM cannot see, or a registry value that a prior partial removal already cleared — the IME concludes there is nothing to remove and reports success **without touching the app**. The removal never happened.

Checks:

1. Read the detection rule: **Apps > All apps > _<your app>_ > Properties > Detection rules**. Note the rule type (MSI / File / Registry / Custom script) and its exact target.

1. On the device, verify the rule against reality with `APPUNIN-Discovery.ps1`:

- **MSI ProductCode** — does the code the rule expects actually appear under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall` (or the WOW6432Node view)? If the app was upgraded, the installed ProductCode may differ from the one in the detection rule.

• **File** — does the file exist at the exact path the rule checks, including `Program Files` vs `Program Files (x86)`, and does the **version operator** still match?

• **Registry** — does the key/value exist, and is the **32-bit registry view** checkbox set to match where the app actually writes?

• **Custom script** — run it manually **as SYSTEM** (`PsExec -s -i powershell`) and confirm it prints to STDOUT and exits 0 only when the app is present.

1. In the log, find the pre-uninstall detection line for the app id. If it says *not applicable / not detected* and there is **no** uninstall command line after it, this is your branch.

Exact fix: Correct the detection rule so it **truthfully reports the app as present** while the app is installed. Detection must match the software's real footprint — author it from `APPUNIN-Discovery.ps1` output, not from vendor docs. Save with **Review + save**. Now the next cycle will detect the app, run the uninstall command, and actually remove it.

Re-validate: Sync, wait a cycle, re-run `APPUNIN-Validation.ps1`, and confirm on the device that the software is genuinely gone (Gate 1 in `05-testing.md`) — not merely that Intune says so.

Branch C — Uninstall runs forever / never reports done (detection still matches after removal)

Symptom: The app **is** removed on the device, but Intune reports **Uninstall failed** or keeps **re-running the uninstall** on every 8-hour cycle. The log shows a **success** exit code from the uninstall command, followed by a **detection = detected** line.

Why it happens: After the uninstall command returns success, the IME re-runs the **same detection rule**. The removal is only reported successful when detection now returns **not detected**. If the uninstaller leaves a harmless remnant that the detection rule still matches — a registry key under `Uninstall`, a leftover DLL, a config folder, or a version operator that is too loose ("greater than or equal to 1.0") — detection still says **detected**, so Intune believes the removal failed and tries again indefinitely.

Checks:

1. On the device, after the uninstall has run, use `APPUNIN-Discovery.ps1` to see what the uninstaller left behind — the exact leftover key/file the detection rule is still catching.

1. Compare that remnant to the detection rule (**Properties > Detection rules**). A file rule that points at a log or config file the uninstaller does not delete is the classic case.

1. In the log, confirm the sequence: uninstall command → exit **0** → detection **detected** → status **failed**. That proves detection, not the uninstaller, is the problem.

Exact fix: Make the detection rule test the **primary artifact the uninstaller actually removes** — usually the app's main EXE with a version operator, or the MSI ProductCode — **not** a remnant the uninstaller leaves behind. Options, in order of preference:

• Repoint the detection rule at the executable/ProductCode that *is* removed.

• If you must use a registry rule, target the specific value the uninstaller clears, and set the operator so it fails once the app is gone.

• As a last resort, add a small step to the removal to delete the specific remnant (see Branch E's wrapper approach), then keep detection pointed at it.

Save with **Review + save**.

Re-validate: Sync, wait a cycle, re-run `APPUNIN-Validation.ps1` **twice** (two cycles) — the state must settle on **Not installed** and **stay there** on the second cycle (no re-attempt). That stability is Gate 4.

Branch D — "Uninstall failed (1605)" (wrong or stale MSI ProductCode)

Symptom: Intune reports **Uninstall failed** with code **1605**. The log shows the uninstall command `msiexec /x {GUID} /qn` ran and Windows Installer returned **1605**.

Why it happens: 1605 = "This action is only valid for products that are currently installed." The **ProductCode GUID** in your uninstall command does not match any product Windows Installer knows about on this device. This happens when the app was **upgraded** and the new version shipped a **different ProductCode** (a "major upgrade"), while the Intune uninstall command still carries the **old** ProductCode — or when the app was installed by a different mechanism and never registered that ProductCode at all.

> ***ProductCode drift is normal.*** *A vendor changes the ProductCode on major-version upgrades but keeps the same* `UpgradeCode`*. An uninstall command hard-coded to v3's ProductCode fails with 1605 against a device now running v4.*

Checks:

1. On the device, run `APPUNIN-Discovery.ps1`. It lists the **actual installed ProductCode(s)** for the app from the registry, and the `UninstallString` each registered. Compare the ProductCode in your Intune **Uninstall command** to the one actually installed.
1. Note that different devices may carry **different** ProductCodes if they are on different versions — a single hard-coded GUID cannot remove all of them.
1. Confirm 1605 in the log against the exact `msiexec /x {GUID}` line.

Exact fix (choose one):

• **If the fleet is on a single version:** update **Properties > Program > Uninstall command** to the **correct current ProductCode** — `msiexec /x {CORRECT-GUID} /qn /norestart`. **Review + save.**

• **If devices span multiple versions:** do **not** hard-code a single GUID. Set the uninstall command to a small wrapper that **resolves the ProductCode by UpgradeCode (or DisplayName) at run time** and then calls `msiexec /x`. `APPUNIN-Remediation.ps1` can generate this wrapper and set it as the uninstall command. Example logic the wrapper uses:

```
# Resolve the installed ProductCode for the app, then uninstall it silently.
$keys = 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
        'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*'
$app = Get-ItemProperty $keys -ErrorAction SilentlyContinue |
       Where-Object { $_.DisplayName -like 'Contoso LOB Suite*' } | Select-Object -First 1
if ($app.PSChildName -match '^\{[0-9A-Fa-f-]{36}\}$') {
```

```
    Start-Process msiexec.exe -ArgumentList "/x $($app.PSChildName) /qn /norestart" -Wait
}
```

Re-validate: Sync, wait a cycle, re-run `APPUNIN-Validation.ps1`. The 1605 must be gone and the state must move to **Not installed** across devices on every version.

Branch E — "Uninstall failed (1603 / other non-zero)" (wrong uninstall command or EXE switches)

Symptom: Intune reports **Uninstall failed** with **1603** (fatal MSI error) or a vendor-specific non-zero code. The log shows the uninstall command ran and returned that code.

Why it happens: The uninstall command itself is wrong for this installer:

- **EXE installers** frequently use a **different silent switch for uninstall than for install** —

e.g. install is `setup.exe /S` but uninstall is `uninstall.exe /S` or `setup.exe /uninstall /quiet`. Reusing the install switches (or omitting the silent switch, so the uninstaller shows UI that SYSTEM cannot interact with, then times out) fails.

- **Quoting/escaping errors** — a path with spaces that is not quoted, or nested quotes mangled in

the command field, so the wrong executable or arguments are passed.

- **1603** specifically is a generic fatal MSI failure — a custom action failed, a file was locked,

or the uninstall needed rights it did not have.

Checks:

1. Read **Properties > Program > Uninstall command** and compare it to the registered

`QuietUninstallString` from `APPUNIN-Discovery.ps1`. Prefer the vendor's documented **silent** uninstall switch.

1. **Reproduce by hand on the device, as SYSTEM.** Use `PsExec -s -i cmd`, then run the exact

uninstall command. Watch whether it (a) shows a UI dialog (means the silent switch is wrong), (b) errors immediately (means the path/switch is wrong), or (c) succeeds (means the problem is context — go to Branch F/G).

1. For **1603** from MSI, re-run with **verbose logging**:

`msiexec /x {GUID} /qn /norestart /l*v C:\ProgramData\APPUNIN\Evidence\uninstall.log`, then open the log and search bottom-up for **"return value 3"** — the line just above it names the failing action.

Exact fix:

- Set **Uninstall command** to the **correct silent uninstall** string for this installer, quoting

any path with spaces: e.g. `"C:\Program Files\Contoso\uninstall.exe" /S` or `"%ProgramFiles%\Contoso\setup.exe" /uninstall /quiet /norestart`.

- If the vendor uninstaller cannot run truly silently, wrap it: package a tiny PowerShell that calls

the uninstaller and forces the silent flags, and set **that** as the uninstall command.

- For 1603 caused by a locked file or a leftover custom action, resolve the specific action from the

verbose log (often a running process — go to Branch G) and retry.

Re-validate: Reproduce the corrected command by hand as SYSTEM first (must exit **0**), then set it in Intune, sync, and confirm **Not installed** via `APPUNIN-Validation.ps1`.

Branch F — Works for some, fails for others (per-user vs per-machine / SYSTEM context)

Symptom: The same Uninstall assignment removes the app from some devices/users but not others. The app is registered under **HKCU** on the failing machines, or was installed "for the current user."

Why it happens: Win32 app uninstall commands run in the **Install behavior** context set on the app — **System** or **User**. A **per-user** app (registered under `HKCU\Software\Microsoft\Windows\CurrentVersion\Uninstall`, installed into the user's profile) is **invisible to SYSTEM**: when the IME runs the uninstall as SYSTEM, it cannot see the per-user registration or the per-user ProductCode, so removal does nothing or fails. Conversely a per-machine app cannot be removed by a user-context command that lacks rights. The mismatch between where the app is registered and the context the uninstall runs in is the root cause.

Checks:

1. Run `APPUNIN-Discovery.ps1` — it reports whether the app is registered under **HKLM**

(per-machine) or **HKCU** (per-user), and under which user's hive.

1. Read **Properties > Program > Install behavior** — **System** or **User**.

2. In the log, a SYSTEM-context uninstall of a per-user app shows detection/uninstall finding

nothing because it is looking in the machine hive.

Exact fix:

• If the app is genuinely **per-user**, the uninstall must run in **User** context, **or** the

command must enumerate loaded user hives and uninstall per user. Simplest correct path: set **Install behavior = User** so the command runs in the signed-in user's context and can see the `HKCU` registration. (Note: user-context Win32 apps only act when that user is signed in.)

• If the app should be **per-machine**, the real fix is upstream — repackage/reinstall it

per-machine so a SYSTEM uninstall can remove it fleet-wide; a per-user install can never be cleanly removed for all users by a single SYSTEM command.

• For a one-time cleanup of an existing per-user install, `APPUNIN-Remediation.ps1` can generate a

SYSTEM wrapper that loads each user hive (`reg load`) and runs the per-user uninstall string — flag this as advanced and test on the pilot first.

Re-validate: Confirm removal on a device of each affected context (a per-user-install machine and a per-machine machine) before expanding.

Branch G — "Uninstall failed (1618)", file-in-use, or reboot required (app running / pending reboot)

Symptom: Intune reports **Uninstall failed (1618)**, or the verbose MSI log shows *files in use*, or removal only succeeds after a reboot. The app has a **running process** or a **loaded service** at uninstall time.

Why it happens:

- **1618 = "Another installation is already in progress."** Windows Installer serializes; if another MSI transaction (or the app's own updater) is mid-flight, the uninstall cannot get the installer mutex and fails. It usually succeeds on the next attempt.
- **Files/services in use** — the app's EXE, a tray helper, or a Windows service is running, so the uninstaller cannot delete the files and either fails (1603) or requires a reboot to complete (**3010** soft-reboot, **1641** hard-reboot).

Checks:

1. Run `APPUNIN-Discovery.ps1` — it lists the app's **running processes** and **services** on the device at the moment you check.
1. Read the exit code: **1618** (retry), **3010** (success, reboot needed to finish), **1641** (success, uninstaller initiated a reboot).
1. In the verbose MSI log, search for **"in use"** / the specific file the uninstaller could not remove.

Exact fix:

- **For 1618:** map **1618** in **Properties > Return codes** as **Retry** (not Failure). Intune will re-attempt on the next cycle when the installer mutex is free. Do not treat 1618 as a real failure.
- **For files in use:** ensure the uninstall command **stops the app first**. Wrap the uninstall so it stops the app's service and kills its processes before calling the uninstaller — e.g. `Stop-Service 'ContosoSvc' -Force; Stop-Process -Name 'Contoso*' -Force; & msiexec /x {GUID} /qn`. `APPUNIN-Remediation.ps1` generates this wrapper.
- **For reboot codes:** map **3010** and **1641** as **success (soft/hard reboot)** in **Properties > Return codes** so a legitimate reboot-to-finish is not reported as a failure, and let the machine reboot to complete file deletion.

Re-validate: After the fix (and a reboot where required), sync and confirm the state settles on **Not installed** and stays there.

Branch H — Superseded app won't come off during an upgrade (supersedence uninstall command)

Symptom: You configured **supersedence** with **Uninstall previous version = Yes**, but the old app stays installed, or the new app is blocked because the old one won't leave. The log references the **superseded** app id and a failing uninstall.

Why it happens: Supersedence with "uninstall previous version" runs the **superseded (old) app's own Uninstall command**. If that old app object's uninstall command is wrong — a stale ProductCode (Branch

D), the wrong EXE switch (Branch E), or a running-process problem (Branch G) — the supersedence removal fails, and depending on settings the new app may not install. The failure is in the **old** app's definition, which people forget to look at because they are focused on the new one.

Checks:

1. Identify the superseded app: **Apps > All apps > _<new app>_ > Properties > Supersedence** — note

the older app it references and whether **Uninstall previous version** is **Yes**.

1. Open the **superseded (old)** app and diagnose **its** Uninstall command exactly as in Branches

C–G — read **its** Program > Uninstall command, test it by hand on a device, decode its exit code.

1. Confirm the supersedence chain is **a single hop** — chains deeper than one hop are unsupported and

stall; run `APPUNIN-Validation.ps1 -CheckConflicts` which also reports supersedence depth.

Exact fix: Correct the **superseded app's** Uninstall command (apply the matching branch: D for ProductCode, E for switches, G for running processes). Flatten any supersedence chain deeper than one hop. Then let the new app's supersedence re-run.

Re-validate: On a pilot device that has the old version, assign the new (superseding) app and confirm the old one is removed and the new one installs, with detection settling correctly for both.

Branch I — Device never reports / state "Unknown" / app object deleted mid-removal (reporting & lifecycle)

Symptom: The device shows **Unknown** or never updates its uninstall status; or you **deleted the app object** in Intune to "force" removal and now the software is stranded on devices with no way to target it.

Why it happens:

• **Stale check-in.** The device has not synced — the IME reports removal status only on its cycle or

a manual sync. A device that is offline, powered off, or has a broken IME never reports, so the console shows **Unknown** or a stale state.

• **App object deleted before removal completed.** Deleting the Win32 app in Intune removes the

assignment and the ability to target it — it does **not** uninstall the app from devices that had it. Any device that had not yet processed the Uninstall intent when you deleted the object is now **orphaned**: the software stays, and there is no app object left to assign an Uninstall intent to. This is a common self-inflicted wound.

Checks:

1. Confirm the device's last check-in: **Devices > Windows > Windows devices > _<device>_** — read

Last check-in time. If it is days old, the device is not reporting, not failing.

1. Run `APPUNIN-Report.ps1` — it summarizes, per app, how many devices are `notInstalled`,

`uninstallFailed`, `pendingInstall`, and `unknown`, so you can separate genuine failures from non-reporting devices.

1. For a deleted app object, check whether the software is still present on devices with

`APPUNIN-Discovery.ps1` — if it is, you have orphaned installs.

Exact fix:

- **For stale check-in:** force a sync (see below) or have the user connect the device; if the IME is

broken, restart the **Microsoft Intune Management Extension** service or re-sync from **Devices > _<device>_ > ... > Sync**. Give it a full cycle before judging it failed.

- **For a deleted app object / orphaned installs: re-create the Win32 app** with the same package

and, crucially, a **correct Uninstall command and detection rule**, assign it as **Uninstall** to `APPUNIN – Uninstall – Production`, and let the IME remove it properly. Do **not** delete an app object as a removal method — always assign **Uninstall** and confirm removal *before* deleting the object.

Re-validate: `APPUNIN-Report.ps1` should show the `unknown`/`uninstallFailed` counts fall to zero as devices check in and process the correctly-configured Uninstall intent.

After any fix — force a sync and re-validate the whole removal

Do not stop at the branch you fixed. On the device, force a sync one of these ways:

- **On the device:** Settings > **Accounts** > **Access work or school** > (the work account) >

Info > **Sync**.

- **From Intune: Devices > Windows > Windows devices > _<device>_ > ... > Sync**.
- **From the toolkit:** `APPUNIN-Remediation.ps1 -AppNameLike "<your app>" -Action Resync` (defaults

to `-WhatIf`; add `-WhatIf:$false` to apply).

Wait a full cycle, then run `APPUNIN-Validation.ps1` and walk the four gates in `05-testing.md`. Only when all four pass on `APPUNIN – Uninstall – Pilot` should you move the Uninstall assignment to `APPUNIN – Uninstall – Production`.

Testing

Use these gates to prove a Win32 app is **actually removed** — not merely that Intune says so, and not merely that it left one stage. Removal is only "fixed" when it passes **all four** gates on a device in `APPUNIN – Uninstall – Pilot`. Run `APPUNIN-Validation.ps1` to read state from Graph and `APPUNIN-Discovery.ps1` on the device to confirm the on-disk reality, and check both against the IME log.

The gates deliberately test the two things the console alone cannot tell you: that the software is **physically gone from the device**, and that Intune **stops trying to remove it**.

Gate 1 — Gone from the device (the software is physically removed)

What it proves: the uninstall command actually ran and removed the app — the console's word is not enough (Branch B fails here even when Intune reports success).

Pass:

- On the device, `APPUNIN-Discovery.ps1` finds **no** registered ProductCode / UninstallString for the app under either `HKLM` or `HKCU`.
- The app's install directory and primary executable are gone.
- No leftover service or scheduled task for the app remains.

Fail: The app is still registered or its files remain → the uninstall never ran (**Branch B**: detection said "gone" pre-emptively) or the command failed (**Branch D/E/G**).

Gate 2 — Uninstall command returned success

What it proves: the uninstall command executed and Windows Installer / the vendor uninstaller returned a success (or legitimate reboot) code.

Pass:

- `AppWorkload.log` shows the uninstall command line running and returning **0**, **1707**, **3010** (soft reboot), or **1641** (hard reboot).
- Any code the vendor documents as success is **mapped as success** in **Properties > Return codes**.

Fail:

- **1605** → wrong/stale ProductCode → **Branch D**.
- **1603** or vendor non-zero → wrong command/switches → **Branch E**.
- **1618** or files-in-use → app running / installer busy → **Branch G**.

Gate 3 — Intune agrees it is gone (state = Not installed)

What it proves: post-uninstall detection returned **not detected**, so Intune reports the removal as successful.

Pass:

- Intune → **Apps > All apps > _<your app>_ > Device install status** shows the device as **Not installed**.
- `APPUNIN-Validation.ps1` reports `installState = notInstalled` for the device.
- The IME log shows the post-uninstall detection rule returning **not detected**.

Fail:

- Reports **Uninstall failed** with a success exit code just above it → detection still matches a remnant → **Branch C**.

- Reports **Installed** with no uninstall attempt → intent conflict → **Branch A**.

Gate 4 — It stays gone (no re-attempt on the next cycle)

What it proves: the removal is stable — Intune is not stuck in a re-uninstall loop, and the app does not reappear.

Pass:

- Run `APPUNIN-Validation.ps1` again after **a second** sync cycle: the device is still

Not installed, and the IME log shows **no** new uninstall attempt for the app on that cycle.

- `APPUNIN-Report.ps1` shows the app's **uninstallFailed** count at **0** and the **notInstalled**

count rising as the pilot completes.

Fail: The uninstall runs again on the next cycle → detection loop → **Branch C**; or the app reappears → an intent conflict re-installed it → **Branch A**.

Full-removal regression check (run on the pilot before expanding)

1. Confirm the device is in `APPUNIN – Uninstall – Pilot` and **not** in any Required/Available group

for the app (`APPUNIN-Validation.ps1 -CheckConflicts`).

1. Force a sync (`APPUNIN-Remediation.ps1 -Action Resync -WhatIf:$false`, or the device **Sync**

button).

1. After one cycle, run `APPUNIN-Validation.ps1` and `APPUNIN-Discovery.ps1` — confirm **Gates 1–3**.
2. After a **second** cycle, re-run both — confirm **Gate 4** (stable, no re-attempt).
3. Run `APPUNIN-Report.ps1` and confirm the pilot group's **notInstalled** count is complete and

uninstallFailed is zero.

Only when **all four gates** pass on the pilot should you move the Uninstall assignment from `APPUNIN – Uninstall – Pilot` to `APPUNIN – Uninstall – Production`.

Troubleshooting

This guide covers problems with the **method, scripts, and access** used in this solution. To diagnose an actual removal failure, use the **Diagnostic Playbook** (`03-implementation.md`) — that is the symptom tree for the apps themselves. This file is about getting the tooling and the diagnostic process to work.

Connecting to Microsoft Graph

`Connect-MgGraph` **fails or the module is missing.**

- Install the SDK: `Install-Module Microsoft.Graph -Scope CurrentUser`.
- Confirm it is present: `Get-Module Microsoft.Graph -ListAvailable`.
- Run PowerShell as your admin account and re-run the script; consent to the requested scopes at the

sign-in prompt.

"Insufficient privileges" or consent is required.

- The read-only scripts need `DeviceManagementApps.Read.All`,

`DeviceManagementManagedDevices.Read.All`, and (for `-CheckConflicts`) `Group.Read.All`. `APPUNIN-Remediation.ps1` needs `DeviceManagementApps.ReadWrite.All`.

- If your tenant blocks user consent, have a Global Administrator grant admin consent once for the

Microsoft Graph Command Line Tools app at **Entra admin center > Applications > Enterprise applications**.

US Government tenant — the scripts connect to the wrong cloud.

- Pass `-GraphEnvironment USGov` (GCC High) or `-GraphEnvironment USGovDoD` (DoD). The script then

calls `Connect-MgGraph -Environment USGov/USGovDoD` and targets `https://graph.microsoft.us` (DoD: `https://dod-graph.microsoft.us`). The admin console is `https://endpoint.microsoft.us`.

- If a script still reaches the Commercial cloud, run `Disconnect-MgGraph` first — a cached Commercial

context is being reused — then re-run with the `-GraphEnvironment` flag.

Scripts return no data even though the app exists.

- Confirm your account has an Intune role that can read apps (R1) and that you signed in to the

correct tenant. Re-run `Connect-MgGraph -TenantId <your-tenant-id>` if you belong to multiple tenants.

Running the on-device scripts

`APPUNIN-Discovery.ps1` **finds nothing, but the app is clearly installed.**

- Run PowerShell **elevated** — a per-machine registration under HKLM is readable without elevation,

but the process/service checks and the `C:\Windows\Installer` scan need admin.

- The app may be registered only under **HKCU** for a *different* user's profile than the one you are

signed in as. That is Branch F — the app is per-user.

- If the app truly has no registration yet Intune reports it installed, the detection rule is matching

a remnant (Branch C).

`APPUNIN-LogParser.ps1` **reports "no matching lines."**

• Match by the application **GUID** (`-AppId <guid>`) as well as the name — the log often references the app id, not the display name.

• The app may never have reached the device (intent conflict — Branch A — or the device is not reporting — Branch I). Confirm with `APPUNIN-Validation.ps1` first.

• Older entries roll into numbered log copies; the parser already includes them, but confirm your `-LogPath` is correct if the IME logs were relocated.

Reading the IME logs

The Intune Management Extension logs are not where expected.

• Path is `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\` — `ProgramData` is hidden; type the full path into File Explorer's address bar, or use `CMTrace`/Notepad.

• Chief files: `IntuneManagementExtension.log`, `AppWorkload.log`, and `AgentExecutor.log`. Older entries roll into numbered copies — check those for a removal that happened earlier.

No log entries for the app at all.

• The device may never have been targeted with the Uninstall intent (Branch A conflict, or a deleted app object — Branch I). Confirm assignments with `APPUNIN-Validation.ps1 -CheckConflicts` first.

• Force a sync and re-check after the next cycle.

Cannot access the device locally.

• Use **Intune admin center > Devices > Windows > Windows devices > _<device>_ > ... > Collect diagnostics** to package the Management Extension logs, then read them offline.

Interpreting states

Console and script disagree on status.

• The console can lag; Graph (via `APPUNIN-Validation.ps1`) reflects the last reported state. Force a sync and re-read; treat the **device log** as the final authority.

Status keeps flipping between Not installed and Uninstall failed.

• This is the detection-loop symptom — go to Diagnostic Playbook **Branch C**. It is not a tooling problem.

Everything shows "Unknown."

• The devices are not checking in, not failing — Branch I. Confirm **Last check-in time** on the

device before chasing uninstall-command fixes.

Remediation script

Remediation made no change.

- By design: `APPUNIN-Remediation.ps1` defaults to **-WhatIf** and only prints the planned change.

Re-run with `-WhatIf:$false` after reviewing it to apply.

Remediation cannot modify the app.

- Read-only rights are enough for Discovery/Validation/Report/LogParser but not for changes. Assign

Intune Administrator (or **Managed apps** Update/Assign) per requirement R2 and re-consent to the ReadWrite scope.

The corrected uninstall command is set in Intune but the device hasn't changed.

- Devices apply on their sync cycle. Force a sync and allow a full cycle, then re-run

`APPUNIN-Validation.ps1`. Do not re-create the app or loop remediation tightly — read the log to confirm the app processed the new command.

The ProductCode-resolving wrapper does not remove the app.

- Confirm the `-DisplayNamePattern` actually matches the app's registered **DisplayName** on the

device (check with `APPUNIN-Discovery.ps1`). A pattern that is too specific (an exact version string) will not match after an upgrade.

When the method stalls

- **You fixed a stage and it still fails.** You likely fixed a later stage while an earlier one is

still wrong. Return to Step 0, re-read the log in removal order (**intent** → **pre-detection** → **command** → **exit code** → **post-detection**), and fix the **first** thing that is wrong — usually the intent conflict (Branch A) or the detection rule (Branch B/C).

- **Removal works for some devices, not others.** Compare a passing device's `APPUNIN-Discovery.ps1`

output to a failing one — the difference (app version / ProductCode, HKLM vs HKCU, a running process, cached-MSI presence) is your cause.

- **The software is stranded because the app object was deleted.** Re-create the Win32 app with a

correct Uninstall command and detection rule and assign **Uninstall** (Branch I) — never delete the object as a removal method.

Best Practices

These practices stop the removal failures the Diagnostic Playbook exists to fix. Apply them when you first **package and assign** an app, and most removals later just work.

Author the uninstall command at package time — not at removal time

• **Record the correct silent uninstall command the day you package the app**, and store it in the app's **Properties > App information > Notes**. The worst time to discover the right switch is during an emergency fleet removal.

• **For MSI apps, never hard-code a single ProductCode if the app will ever be upgraded.** Major upgrades change the ProductCode. Prefer an uninstall command that **resolves the ProductCode by UpgradeCode or DisplayName at run time** (the wrapper in Branch D), so removal survives version drift across the fleet.

• **For EXE apps, get the silent-uninstall switch from the vendor's documentation** and confirm it — install and uninstall switches are frequently different. Test the uninstall by hand, as SYSTEM, and confirm it exits **0** with no UI.

• **Quote every path with spaces** in the uninstall command, and prefer environment variables (`%ProgramFiles%`) over hard-coded drive paths.

Detection rules decide removal — make them match reality on both ends

• **Detect the primary artifact the uninstaller removes** — the main executable (with a version operator) or the MSI ProductCode — **not** a config file, log, or registry key the uninstaller leaves behind. A remnant-based rule causes the infinite re-uninstall loop (Branch C).

• **Set the version operator deliberately.** "Greater than or equal to" survives auto-updates for *install* detection, but for a clean removal make sure the artifact disappears entirely so detection flips to not-detected.

• **Mind 32-bit vs 64-bit.** Point file rules at the correct `Program Files` / `Program Files (x86)` location and set the **32-bit registry view** checkbox to match where the app writes — or SYSTEM detection will look in the wrong hive (Branch B/F).

• **Test custom detection scripts as SYSTEM.** They must print to STDOUT and exit 0 only when the app is present. Run them with `PsExec -s` before shipping.

Keep intents unambiguous — never target install and uninstall together

• **A device must never be in both a Required/Available group and an Uninstall group for the same app. `Required` wins, and the uninstall silently does nothing (Branch A). Maintain removal through the dedicated** Assigned** group `APPUNIN – Uninstall – Production`, and ensure those devices are not members of any install group for that app.

• **Watch nested groups.** A device can be in a Required group transitively. Resolve effective

membership (`APPUNIN-Validation.ps1 -CheckConflicts`) before you trust an assignment.

- **Use** `APPUNIN – Uninstall – Exclusions` for the devices that must keep the app (break-fix, labs,

executives), and exclude that group from the Uninstall assignment.

Map return codes so benign outcomes are not failures

- **Map 3010 and 1641 as success (soft / hard reboot).** A removal that needs a reboot to finish

deleting files is a success, not a failure.

- **Map 1618 as Retry**, not Failure — it just means Windows Installer was busy.

- **Do not map 1605 or 1603 as success** to hide a problem — those mean the removal genuinely did not

happen; fix the command (Branch D/E) instead.

Stop the app before removing it

- **For apps with a running process or service, wrap the uninstall to stop them first**

(`Stop-Service`, `Stop-Process`) so the uninstaller is not blocked by files in use (Branch G).

- **Prefer per-machine installs.** A per-user install can never be cleanly removed for all users by a

single SYSTEM uninstall (Branch F); if you control the packaging, install per-machine.

Remove the right way — assignment first, delete the object last

- **Never delete the Intune app object to "force" removal.** Deleting the object removes your ability

to target the app; devices that had not yet processed the Uninstall intent are orphaned with the software still on them (Branch I). Always assign **Uninstall**, confirm removal on the fleet, and only then delete the object.

- **Pilot every removal.** Assign Uninstall to `APPUNIN – Uninstall – Pilot` first, pass all four

gates in `05-testing.md`, then expand to `APPUNIN – Uninstall – Production`. Removal is destructive and often not reversible without data loss.

Operational hygiene

- **Schedule the read-only reporter.** Run `APPUNIN-Report.ps1` regularly to catch rising

`uninstallFailed` counts before users notice software that should be gone.

- **Read the IME log before changing settings.** `IntuneManagementExtension.log`,

`AppWorkload.log`, and `AgentExecutor.log` are authoritative for intent, exit code, and detection — check them first.

• **Keep remediation gated.** `APPUNIN-Remediation.ps1` defaults to **-WhatIf**; review the planned change, then re-run with `-WhatIf:$false` to apply.

Deliverables Summary

This solution delivers a complete, repeatable method plus tooling for diagnosing and fixing **Win32 app uninstall / removal failures** in Microsoft Intune.

Documents

Document	What it gives you
Solution Overview (`01-solution-overview.md`)	How Intune processes an Uninstall intent (intent reconciliation, detection-gated removal, exit-code mapping, install context), the diagnostic model, and the named objects the solution creates.
Global Requirements (`global-requirements-intune-solutions.md`)	Category-level prerequisites (tenant, Intune, enrollment, Windows baseline, network).
Requirements (`02-requirements.md`)	Solution-specific prerequisites — Intune roles, Graph scopes, device + IME-log access, the original packaging record, a pilot group — each with Description / Why / Verify / Implement, plus the US Government (GCC High / DoD) targeting note.
Diagnostic Playbook (`03-implementation.md`)	Step 0 fact-finding plus nine removal-failure branches (A–I): intent conflict, detection-said-gone, detection loop, stale ProductCode (1605), wrong command/switches (1603), per-user vs per-machine context, app-in-use (1618), supersedence removal, reporting/lifecycle — each with symptom → why → exact fix → re-validate.
Testing (`05-testing.md`)	The four removal gates: gone from the device, uninstall returned success, Intune agrees, and it stays gone — plus a full-removal regression check.
Troubleshooting (`06-troubleshooting.md`)	Fixes for problems with the method, scripts, and access themselves.
Best Practices (`07-best-practices.md`)	Package-time discipline (uninstall command, detection rules, unambiguous intents, return-code mapping) that prevents removal failures.
Executive Summary (`executive-summary.md`)	The business case in one page.

PowerShell scripts

All Graph-based scripts use the Microsoft Graph PowerShell SDK and accept `-GraphEnvironment Global | USGov | USGovDoD` for US Government tenants. Discovery, Validation, Report, and

LogParser are strictly **read-only**. Remediation defaults to **-WhatIf** — it shows the planned change and makes no modification unless you re-run with `-WhatIf:$false`.

Script	Purpose	Access
APPUNIN-Discovery.ps1	On the device: resolve the real MSI **ProductCode**, the registered **UninstallString**/**QuietUninstallString**, HKLM vs HKCU registration, whether the cached MSI still exists in `C:\Windows\Installer`, and any running app processes/services.	Read-only (local)
APPUNIN-Validation.ps1	Per-device uninstall/detection state for an app from Graph, mapped to the four gates; with `-CheckConflicts`, flags devices targeted by both an install and an uninstall intent and reports supersedence depth.	Read-only (Graph)
APPUNIN-Report.ps1	Fleet-wide removal report — per app, counts of notInstalled / uninstallFailed / pendingInstall / unknown, to separate real failures from non-reporting devices.	Read-only (Graph)
APPUNIN-LogParser.ps1	Parse the IME logs (`IntuneManagementExtension.log`, `AppWorkload.log`, `AgentExecutor.log`) for an app and print the uninstall sequence: resolved intent, pre-detection, uninstall command, exit code, post-detection.	Read-only (local)
APPUNIN-Remediation.ps1	Guarded fixes: correct an app's Uninstall command (including a ProductCode-resolving wrapper), correct a detection rule, remove a conflicting assignment, or force a device re-sync.	-WhatIf default (Graph)

How the pieces fit

1. Confirm **Requirements**.

2. Run **Step 0** — read reported state, list assignments (**Validation -CheckConflicts**), read the device (**Discovery**), read the **IME log** (**LogParser**).

1. Follow the **Diagnostic Playbook** branch (A–I) that matches your symptom.
2. Apply the fix; use **Remediation** (gated) to correct the command / detection / assignment.
3. Prove it with the four **Testing** gates on `APPUNIN – Uninstall – Pilot` (via **Validation** +

Discovery).

1. Expand to `APPUNIN – Uninstall – Production` and track fleet health with **Report**, following **Best Practices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPUNIN-Discovery.ps1

Purpose: Runs LOCALLY on a failing Windows device (elevated) to establish the on-disk truth an Intune uninstall depends on. For the target app it resolves: - the real MSI ProductCode(s) actually installed (HKLM and WOW6432Node), - the registered UninstallString / QuietUninstallString, - whether the app is registered per-machine (HKLM) or per-user (HKCU), - whether the cached MSI still exists under C:\Windows\Installer, - any running processes / services owned by the app (which block removal). This is what you compare against the app's Intune Uninstall command and detection rule in Diagnostic Playbook branches B, C, D, E, F, and G. HOW TO RUN Open an ELEVATED PowerShell on the device, then: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Discovery.ps1 -DisplayNameLike "Contoso LOB" Optional: -CsvPath C:\ProgramData\APPUNIN\Evidence\discovery.csv to save the result. No Microsoft Graph and no external modules are required. EXPECTED RESULTS A per-registration table (DisplayName, Version, Scope HKLM/HKCU, ProductCode, UninstallString, CachedMsiPresent) plus a list of running processes/services. Pass = the ProductCode and UninstallString match the Intune Uninstall command. Fail = they differ (Branch D/E), the app is HKCU-only (Branch F), the cached MSI is missing (Branch E last-resort), or a process/service is running (Branch G). .NOTES Intune Solutions — read-only, local. The repeatable way. Makes no changes. >

How to run: Open an ELEVATED PowerShell on the device, then: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Discovery.ps1 -DisplayNameLike "Contoso LOB" Optional: -CsvPath C:\ProgramData\APPUNIN\Evidence\discovery.csv to save the result. No Microsoft Graph and no external modules are required. EXPECTED RESULTS A per-registration table (DisplayName, Version, Scope HKLM/HKCU, ProductCode, UninstallString, CachedMsiPresent) plus a list of running processes/services. Pass = the ProductCode and UninstallString match the Intune Uninstall command. Fail = they differ (Branch D/E), the app is HKCU-only (Branch F), the cached MSI is missing (Branch E last-resort), or a process/service is running (Branch G). .NOTES Intune Solutions — read-only, local. The repeatable way. Makes no changes. >

APPUNIN-LogParser.ps1

Purpose: Runs LOCALLY on the device. Reads the IME logs and extracts, for the target app, the uninstall pipeline in order: the intent the IME resolved (watch for Required beating Uninstall -> Branch A), the pre-uninstall detection result, the uninstall command line that ran, the returned EXIT CODE, and the post-uninstall detection result. This is the definitive evidence for which branch you are in; Graph reports the state, these logs report the reason. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPUNIN-LogParser.ps1 -AppNameLike "Contoso LOB" Optional: -LogPath 'C:\ProgramData\Microsoft\IntuneManagementExtension\Logs' (default) -AppId '<application GUID>' (match by id instead of/as well as name) -OutFile 'C:\ProgramData\APPUNIN\Evidence\log-extract.txt' No Microsoft Graph and no external modules are required. EXPECTED RESULTS The matched log lines (with timestamps and highlighted exit codes) in pipeline order, and a one-line verdict pointing at the likely branch. Pass = intent Uninstall -> detection detected -> command -> exit 0 -> detection notDetected. Fail = the sequence breaks (e.g. exit 1605 -> Branch D; detection detected after exit 0 -> Branch C; no command after 'not detected' -> Branch B). .NOTES Intune Solutions — read-only, local. The repeatable way. Reads log files only. >

How to run: powershell -ExecutionPolicy Bypass -File .\APPUNIN-LogParser.ps1 -AppNameLike "Contoso LOB" Optional: -LogPath 'C:\ProgramData\Microsoft\IntuneManagementExtension\Logs' (default) -AppId '<application GUID>' (match by id instead of/as well as name) -OutFile 'C:\ProgramData\APPUNIN\Evidence\log-extract.txt' No Microsoft Graph and no external modules are required. EXPECTED RESULTS The matched log lines (with timestamps and highlighted exit codes) in pipeline order, and a one-line verdict pointing at the likely branch. Pass = intent Uninstall -> detection detected -> command -> exit 0 -> detection notDetected. Fail = the sequence breaks (e.g. exit 1605 -> Branch D; detection detected after exit 0 -> Branch C; no command after 'not detected' -> Branch B). .NOTES Intune Solutions — read-only, local. The repeatable way. Reads log files only. >

APPUNIN-Remediation.ps1

Purpose: Applies one targeted, reviewed fix to correct a removal failure, per the Diagnostic Playbook. Every action defaults to PREVIEW (-WhatIf): it prints exactly what it WOULD change and makes no modification unless you re-run with -WhatIf:$false. Supported actions: SetUninstallCommand - set Properties>Program>Uninstall command to a known-good string (Branch D/E). Use -GenerateProductCodeWrapper to emit a run-time ProductCode-resolving wrapper for fleets spanning multiple versions (Branch D). Resync - trigger a device sync so the IME re-attempts the pipeline for devices reporting uninstallFailed (post-fix). NOTE: detection-rule and assignment edits are structurally destructive and are intentionally left to the console with the exact click path (printed by this script) so you review them yourself; this tool only writes the uninstall command and triggers re-syncs. HOW TO RUN Preview a corrected uninstall command: .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action SetUninstallCommand -UninstallCommand 'msiexec /x {NEW-GUID} /qn /norestart' Apply it: ... same line ... -WhatIf:$false Emit + set a ProductCode-resolving wrapper (multi-version fleet): .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action SetUninstallCommand -GenerateProductCodeWrapper -DisplayNamePattern "Contoso LOB Suite*" Re-attempt failed devices: .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action Resync -WhatIf:$false US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)

How to run: Preview a corrected uninstall command: .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action SetUninstallCommand -UninstallCommand 'msiexec /x {NEW-GUID} /qn /norestart' Apply it: ... same line ... -WhatIf:$false Emit + set a ProductCode-resolving wrapper (multi-version fleet): .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action SetUninstallCommand -GenerateProductCodeWrapper -DisplayNamePattern "Contoso LOB Suite*" Re-attempt failed devices: .\APPUNIN-Remediation.ps1 -AppNameLike "Contoso LOB" -Action Resync -WhatIf:$false US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)

APPUNIN-Report.ps1

Purpose: For one or all Win32 apps, summarizes per-device uninstall status across the fleet so you can separate GENUINE removal failures from devices that simply have not reported. For each app it counts: notInstalled (removed), uninstallFailed, pendingInstall/pending, installed (still present), and unknown (not reporting). Use it to size a removal campaign, watch it drain, and catch rising uninstallFailed counts before users notice lingering software. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPUNIN-Report.ps1 powershell -ExecutionPolicy Bypass -File .\APPUNIN-Report.ps1 -AppNameLike "Contoso LOB" -CsvPath .\removal-report.csv US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)

How to run: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Report.ps1 powershell -ExecutionPolicy Bypass -File .\APPUNIN-Report.ps1 -AppNameLike "Contoso LOB" -CsvPath .\removal-report.csv US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)

APPUNIN-Validation.ps1

Purpose: Reads, from Microsoft Graph, the per-device install state for a Win32 app and maps it to the four removal gates (gone / uninstall succeeded / Intune agrees / stays gone). With -CheckConflicts it also lists every assignment on the app, resolves group membership, and flags any device that is targeted by BOTH an install intent (Required/Available) AND an Uninstall intent -- the Branch A root cause -- and reports the app's supersedence depth. HOW TO RUN Read state: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Validation.ps1 -AppNameLike "Contoso LOB" Detect intent conflicts + supersedence depth: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Validation.ps1 -AppNameLike "Contoso LOB" -CheckConflicts US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)

How to run: Read state: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Validation.ps1 -AppNameLike "Contoso LOB" Detect intent conflicts + supersedence depth: powershell -ExecutionPolicy Bypass -File .\APPUNIN-Validation.ps1 -AppNameLike "Contoso LOB" -CheckConflicts US Gov: add -GraphEnvironment USGov (GCC High) or -GraphEnvironment USGovDoD (DoD)